

Virtual Worker Platform with n8n and Hugging Face

Executive summary

You can build a “Virtual Worker” platform by separating a **product/control plane** (task intake, tenant management, approvals, audit, billing, model governance) from an **execution plane** (n8n workflows running in scalable queue mode) and an **inference plane** (Hugging Face-hosted or self-hosted models for interpretation, extraction, and tool-parameterization). This separation reduces coupling, improves multi-tenant safety, and lets you scale n8n workers and model servers independently while keeping governance in one place. ¹

For n8n in production, **queue mode with Redis + Postgres** is the recommended baseline because it enables distributed execution (workers) and optional webhook processors, but it requires Redis and a database and is **not supported with SQLite** in distributed queue setups. ²

For Hugging Face inference, you should deliberately choose among three tiers: - **Inference Providers (serverless)** for fastest iteration and broad model access via a unified API. ³

- **Inference Endpoints (dedicated, autoscaling, managed lifecycle including scale-to-zero)** for production workloads needing stable capacity, network controls (e.g., PrivateLink on AWS), and predictable latency/availability. ⁴

- **Self-hosted TGI/TEI** when you need maximum cost control at scale, custom GPU scheduling, or strict data residency/air-gapped deployment (at the cost of SRE burden). ⁵

A production-grade Virtual Worker must be **idempotent**, “retry-safe,” and auditable because workflow executions and external API calls fail in real systems; designing idempotency keys and safe retries up front prevents duplicate side effects (double emails, duplicate CRM updates, double charges). Stripe’s idempotency guidance is a strong reference pattern: store/cached results keyed by an idempotency key and retry with backoff/jitter. ⁶

Multi-tenant isolation is a first-order design requirement: the largest systemic risk is cross-tenant data leakage via bugs/misconfiguration in auth context propagation, storage, caching, or workflow permissions. OWASP’s multi-tenant guidance and Kubernetes’ multi-tenancy guidance both emphasize that sharing infrastructure saves cost but introduces security/noisy-neighbor challenges that must be actively controlled (RBAC, network boundaries, quotas, policy). ⁷

Finally, n8n licensing/edition constraints materially impact the architecture if you intend to **host workflows/credentials for multiple customers**: n8n is “fair-code” under the Sustainable Use License (not OSI “open source”), and embedding/hosting for clients typically requires commercial licensing (Embed / Enterprise), while several governance/security features (Projects/RBAC, External Secrets, External Storage) are not available in Community Edition. ⁸

Open items and decision log

This report assumes no constraints on team size or cloud, as requested. Treat these as **open items** that must be resolved during Phase 0, because they influence security, scalability, and cost:

Team and delivery assumptions (open items): exact team size, whether you have dedicated SRE, and whether “workflow engineers” will build and maintain n8n templates.

Cloud and infra (open items): cloud provider(s), whether Kubernetes is in scope from day one, network design (public vs private), and region/data residency needs.

Customer model (open items): single-tenant deployments per customer vs shared multi-tenant; whether customers bring their own credentials (BYOC) vs you store credentials; and whether you must support regulated industries (SOC 2, ISO 27001, HIPAA-like controls).

Licensing/commercial (critical open item): whether you will (a) embed/white-label n8n, (b) host customer workflows/credentials, and (c) require Enterprise features (External Secrets, External Storage, Projects/RBAC). n8n’s docs and support guidance strongly suggest that “hosting workflows/credentials for clients” requires an Enterprise/commercial arrangement. ⁹

Model and data policy (open items): whether user data can be sent to third-party inference (HF Providers/Endpoints) or must remain in your VPC/cluster; whether prompts/logs are retained; and how you will implement redaction.

Architecture and required components

Architectural overview

At a high level, implement a **three-plane architecture**:

- **Control plane (your product):** multi-tenant API + UI where users submit tasks, choose permissions/approvals, view results, manage connectors and policies, and browse task templates.
- **Execution plane (n8n):** workflow orchestration that executes reliable connector steps (HTTP requests, SaaS nodes, ETL, etc.) at scale using queue mode workers and (optionally) webhook processors. ¹⁰
- **Inference plane (Hugging Face):** model services that interpret tasks, map them to templates, produce structured parameters, and generate text outputs (emails, summaries, reports) via Inference Providers / Inference Endpoints / TGI/TEI. ¹¹

flowchart LR

```
U[User / Company] -->|UI/API Task Intake| FE[Frontend]
FE --> API[Backend API / Control Plane]
```

```
API --> DB[(Postgres: tenants, tasks, approvals, audit)]
```

```
API --> OBJ[(Object Storage: artifacts, attachments)]
```

```
API --> VAULT[Secrets Vault]
```

```

API -->|Plan/Extract/Route| AI[AI Orchestrator Service]
AI --> HF1[HF Inference Providers\n(serverless)]
AI --> HF2[HF Inference Endpoints\n(dedicated)]
AI --> HF3[TGI/TEI Self-hosted\n(GPU/CPU)]

API -->|Dispatch| N8NMAIN[n8n main\n(UI/API)]
N8NMAIN --> REDIS[(Redis Queue)]
REDIS --> N8NWORKERS[n8n workers\n(queue mode)]
N8NMAIN --> N8NWEBHOOK[n8n webhook processors\n(optional)]
N8NWORKERS --> EXT[External Systems\n(Slack, Gmail, CRM, DBs)]
N8NWORKERS --> API

N8NWORKERS --> OBS[Observability\nLogs/Metrics/Tracing]
API --> OBS

```

Key n8n facts to anchor the execution plane: - Queue mode uses Redis as a message broker and persists in the database; SQLite is not supported for distributed queue setups. ²

- Webhook processors are optional for scaling incoming webhook traffic; n8n recommends not putting the main process behind the same load balancer pool to avoid UI degradation. ¹²

- n8n workflows can accept inbound calls via the Webhook node and can respond immediately, when the last node finishes, or via the Respond to Webhook node patterns. ¹³

Required components and recommended technologies

The table below lists **required components** (explicitly requested), recommended technologies, and the “why” behind each.

Component	Recommended technologies	Why needed	Security controls	Cost/ops considerations
Frontend	Next.js/React + TypeScript; Tailwind; OpenAPI-generated client	Task intake UX, approvals, run history, connector setup	Tenant-aware authorization; CSP; rate limits; WAF	Mostly dev cost; CDN hosting; avoid storing secrets client-side
Backend API (Control plane)	Node.js (NestJS/Fastify) or Python (FastAPI); OpenAPI; background queue (BullMQ/Celery)	Owns tenant context, idempotency, approvals, audit, n8n dispatch	Strict tenant context propagation per request; RBAC; signed webhooks; HMAC	Horizontal scale; standardized retry/backoff design reduces incident load ¹⁴

Component	Recommended technologies	Why needed	Security controls	Cost/ops considerations
n8n setup	Self-host n8n in queue mode; Postgres + Redis + workers; optional webhook processors	Reliable workflow orchestration and connector ecosystem; distributed execution	n8n RBAC (Projects) where available; disable unnecessary public API; isolate main vs workers	Queue mode scaling requires Redis + DB; avoid SQLite; scale workers independently ¹⁵
HF inference options	(1) Inference Providers; (2) Inference Endpoints; (3) Self-host TGI (generation) + TEI (embeddings)	Model-backed interpretation, extraction, generation	Private endpoints (AWS PrivateLink) where needed; per-tenant request logging; prompt/ data governance	Providers fastest; Endpoints for stable prod; self-host for cost control but higher SRE burden ¹⁶
Database schema	Postgres	Multi-tenant state: tasks, approvals, idempotency keys, audit logs	Row-level isolation patterns; encryption at rest; DB credentials rotation	Use migrations; partition large tables by time/ tenant if needed
Object storage	S3-compatible (AWS S3 / GCS / Azure Blob / MinIO)	Store artifacts, attachments, generated files, workflow outputs	Pre-signed URLs; per-tenant buckets/prefixes; lifecycle policies	Storage cost centers; enforce retention and lifecycle policies
Secrets vault	HashiCorp Vault (or cloud secret manager); Vault DB secrets engine	Centralize secrets, rotate credentials, limit blast radius	Dynamic secrets with TTL; audit logs; least privilege	Vault adds ops overhead but reduces incident scope; dynamic creds reduce secret sprawl ¹⁷
CI/CD	GitHub Actions; container registry; IaC (Terraform/ Pulumi)	Repeatable builds and deployments	Signed images; SAST/DAST; secrets scanning	Investment pays back quickly in reliability
Monitoring & observability	Prometheus + Grafana; Loki for logs; OpenTelemetry for traces	Detect failures, regressions, tenant issues	Security logging + monitoring is core to forensics and incident response	Scope creep risk—start minimal but standardize early ¹⁸

Hosting trade-offs for Hugging Face inference

Option	What it is	Strengths	Weaknesses	Best fit
HF Inference Providers (serverless)	Unified access to models powered by HF + partners (formerly “Inference API serverless”)	Fastest to start; no infra; broad model choice via unified API + SDK integration ¹⁹	Less control over latency/noisy neighbors; data governance may be harder	Phase 0–1 MVP, low/medium volume, experimentation
HF Inference Endpoints (dedicated)	Managed deployment on autoscaling infra, lifecycle managed by HF (start/stop/scale/health/monitoring)	Dedicated capacity; autoscaling + scale-to-zero; “protected” endpoints; AWS PrivateLink option ⁴	Higher unit cost than serverless; still 3rd-party hosting	Production workloads needing stable SLOs + managed ops
Self-host TGI/TEI	Run HF’s inference servers in your infra: TGI for generation, TEI for embeddings	Full control; can optimize GPU utilization; supports air-gapped/strict residency	You own patching, scaling, GPU scheduling, and incident response	High-volume production, strict compliance, cost optimization ⁵

Implementation blueprints and configuration snippets

n8n deployment blueprint (Docker Compose)

n8n + queue mode requires **Postgres + Redis** and is designed so workers run executions. ²⁰

Docker Compose is the quickest reference packaging format; Docker provides the Compose file reference and `docker compose up` CLI behavior (builds/creates/starts services). ²¹

Example `compose.yaml` (production-ish baseline; add TLS proxy like Caddy/Nginx separately):

```
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: n8n
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: n8n
    volumes:
      - pgdata:/var/lib/postgresql/data
```

```

redis:
  image: redis:7
  command: ["redis-server", "--appendonly", "yes"]
  volumes:
    - redisdata:/data

n8n-main:
  image: docker.n8n.io/n8nio/n8n:latest
  ports: ["5678:5678"]
  environment:
    # Database
    DB_TYPE: postgresdb
    DB_POSTGRESDB_HOST: postgres
    DB_POSTGRESDB_DATABASE: n8n
    DB_POSTGRESDB_USER: n8n
    DB_POSTGRESDB_PASSWORD: ${POSTGRES_PASSWORD}

    # Queue mode (production scaling)
    EXECUTIONS_MODE: queue

    # Secure credentials encryption key (set explicitly!)
    N8N_ENCRYPTION_KEY: ${N8N_ENCRYPTION_KEY}

    # Webhook URL when behind reverse proxy / LB
    WEBHOOK_URL: ${WEBHOOK_URL}

    # Hardening
    N8N_PUBLIC_API_DISABLED: "true" # if you don't use public REST API
    N8N_METRICS: "true"

    # Optional: use Postgres/Redis connection TLS configs in real deployments
    depends_on: [postgres, redis]

n8n-worker:
  image: docker.n8n.io/n8nio/n8n:latest
  command: ["n8n", "worker", "--concurrency=10"]
  environment:
    DB_TYPE: postgresdb
    DB_POSTGRESDB_HOST: postgres
    DB_POSTGRESDB_DATABASE: n8n
    DB_POSTGRESDB_USER: n8n
    DB_POSTGRESDB_PASSWORD: ${POSTGRES_PASSWORD}
    EXECUTIONS_MODE: queue
    QUEUE_BULL_REDIS_HOST: redis
    N8N_ENCRYPTION_KEY: ${N8N_ENCRYPTION_KEY}
    # Concurrency control can be managed via env + flags
    N8N_CONCURRENCY_PRODUCTION_LIMIT: "10"

```

```
    depends_on: [postgres, redis]

volumes:
  pgdata:
  redisdata:
```

Why these settings matter (grounded in docs): - `EXECUTIONS_MODE=queue` puts distributed execution behind Redis + DB; SQLite is not supported for this distributed mode. ²²

- `N8N_ENCRYPTION_KEY` is critical: n8n encrypts credentials before storing them; if you let it auto-generate and then lose the key, you lock yourself out of credentials. n8n documents that it generates a random key on first launch and uses it to encrypt credentials. ²³
- Disabling the n8n public REST API is recommended when unused (`N8N_PUBLIC_API_DISABLED=true`). ²⁴
- Prometheus metrics are disabled by default and enabled via `N8N_METRICS=true`, exposing a `/metrics` endpoint using `prom-client`. ²⁵
- Concurrency control uses `N8N_CONCURRENCY_PRODUCTION_LIMIT` plus worker concurrency flags, which matters for rate limited APIs and stability. ²⁶

Operational commands:

```
# Start stack
docker compose up -d

# View logs
docker compose logs -f n8n-main

# Stop
docker compose down
```

(Compose behavior per Docker CLI docs.) ²⁷

n8n queue mode with webhook processors (Kubernetes or containers)

If you rely heavily on inbound webhooks (task intake, callbacks), n8n supports **webhook processors** as another scaling layer, and recommends not placing the main process in the LB pool (to avoid UI degradation). ¹²

Example command pattern (from n8n docs): webhook processor can run via `n8n webhook` and requires queue mode env. ²²

External storage for large files (n8n binary data → S3)

If your Virtual Worker processes files, you must plan for binary data storage. n8n supports multiple binary data modes and documents that `N8N_DEFAULT_BINARY_DATA_MODE` can be set to `filesystem` or `s3`. ²⁸

n8n's External Storage (S3) capability is an **Enterprise-tier feature**, and n8n delegates pruning to S3 lifecycle rules (so lifecycle configuration is required unless you want to keep binary data forever). ²⁹

Key env vars include: `N8N_EXTERNAL_STORAGE_S3_HOST`, `..._BUCKET_NAME`, `..._BUCKET_REGION`, `..._ACCESS_KEY`, `..._ACCESS_SECRET` and optional `..._AUTH_AUTO_DETECT`. ³⁰

Securing n8n: RBAC, External Secrets, and API keys

For a multi-tenant Virtual Worker, **governance features in n8n can become gating**:

- **RBAC uses Projects** to group workflows/credentials; docs note RBAC isn't available on Community edition. ³¹
- **External secrets** integrate with AWS Secrets Manager, Azure Key Vault, GCP Secrets Manager, Infisical (deprecated), and HashiCorp Vault (but "Vault Secrets" not supported). This is Enterprise-only. ³²
- n8n's public REST API uses API keys via `X-N8N-API-KEY`; enterprise allows scoping, while non-enterprise API keys have full access. ³³

These facts strongly suggest: if your platform requires per-tenant workflow isolation, scoped API keys, external secrets, and S3 binary storage, you should plan for **commercial licensing (Embed/Enterprise)** early. ³⁴

Hugging Face Inference Endpoints: creation, deployment, and private networking

Hugging Face documents that Inference Endpoints provide a "secure production solution" on dedicated autoscaling infrastructure, and the platform manages lifecycle (starting/stopping/scaling/monitoring). ³⁵

Programmatic and CLI examples (from `huggingface_hub` docs): - Create endpoint via Python (`create_inference_endpoint`) and via CLI (`hf endpoints deploy ...`). ³⁶
- You can deploy from a curated catalog via `hf endpoints catalog deploy ...`. ³⁶
- Endpoints can use a **custom container image** (e.g., TGI) with env and health route. ³⁶

Example (CLI):

```
hf endpoints deploy my-endpoint-name \  
  --repo gpt2 \  
  --framework pytorch \  
  --accelerator cpu \  
  --vendor aws \  
  --region us-east-1 \  
  --instance-size x2 \  
  --instance-type intel-icl \  
  --task text-generation
```

(As shown in HF hub docs.) ³⁶

Private networking: - HF documents creating **Private Endpoints with AWS PrivateLink** so traffic does not traverse the public internet. ³⁷

Self-hosted inference: TGI for generation, TEI for embeddings

Hugging Face provides Quick Tours describing how to run **Text Generation Inference (TGI)** via its official Docker container. ³⁸

A credible signal of maturity: the TGI repository describes it as a server “used in production at Hugging Face to power Hugging Chat, the Inference API and Inference Endpoints.” ³⁹

Hugging Face provides Quick Tour documentation for **Text Embeddings Inference (TEI)** with a local HTTP API (e.g., `/embed`), including air-gapped deployment patterns. ⁴⁰

Example: TEI request (from docs):

```
curl 127.0.0.1:8080/embed \
-X POST \
-d '{"inputs":["Today is a nice day","I like you"]}' \
-H 'Content-Type: application/json'
```

⁴¹

Governance, safety, and quality design

Task intake design (UI + API)

Design goals: accept tasks from users/companies, guarantee tenant isolation, make runs repeatable and auditable, and ensure side effects are approved and idempotent.

1) **API-first intake** - `POST /v1/tasks` accepts: `tenant_id`, `user_id`, `task_text`, optional `attachments[]`, optional `constraints` (time window, cost budget, approval policy). - Always include an **Idempotency-Key** (client-generated UUID). Your backend stores the first result for that key and returns the same result for retries (Stripe documents this “store the first result and replay” pattern). ⁴²

2) **UI modes** - “Quick Task” (natural language + optional add-ons). - “Template Task” (structured form bound to a parameter schema). - “Company Task” (requires policy checks, approvals, and connector constraints).

3) **Attachments** - Upload to object storage and pass references (pre-signed URLs) into workflows rather than large binary payloads. - If you store binaries inside n8n, prefer filesystem mode early; for enterprise scaling, S3 mode is supported but requires license and lifecycle policies. ⁴³

AI interpretation (model selection, prompt templates, evaluation metrics)

Core AI jobs in a Virtual Worker: - Intent classification: “what kind of task is this?” - Template selection: “which workflow template matches?” - Parameter extraction: “what are the structured inputs, validated?” -

Tool planning: “what steps will be taken, what credentials needed?” - Generation: draft email/messages/reports.

Recommended model routing approach - Use a small/medium chat model for classification + extraction (cheap, fast), then escalate to larger models only when uncertainty is high or tasks are complex. - Use embeddings for semantic routing (template search, RAG). - Use reranking for better template selection if you have many templates.

Evaluation metrics and harness - For classical metrics and standardized evaluation, Hugging Face provides the `evaluate` library. ⁴⁴

- For LLM evaluation across backends with sample-by-sample debug, Hugging Face provides `lighteval`. ⁴⁵

Practical evaluation KPIs (minimum set) - Template selection: top-1 accuracy, top-3 accuracy. - Parameter extraction: JSON Schema validation pass rate; field-level F1 for key entities. - Execution: task success rate, mean time to completion, approval conversion rate. - Safety: PII leakage incidents, cross-tenant access attempts (should be zero).

Table comparing Hugging Face model options (baseline candidates)

These are practical, widely used baselines on the Hugging Face Hub; you should benchmark against your task distribution.

Capability	Candidate model	License	Notes for Virtual Worker	Serving suggestion
Chat planner / instruction-following	<code>HuggingFaceH4/zephyr-7b-beta</code>	MIT ⁴⁶	Assistant-style model; explicitly warns it can generate problematic text when prompted (plan for safety filters). ⁴⁶	TGI (self-host) or HF Endpoint (custom TGI image) ⁴⁷
Chat planner (function/tool calling)	<code>mistralai/Mistral-7B-Instruct-v0.3</code>	Apache-2.0 ⁴⁸	Includes installation guidance and function calling examples in model card. ⁴⁸	TGI or HF Endpoint
Template semantic search embeddings	<code>BAAI/bge-base-en-v1.5</code>	MIT ⁴⁹	Common retrieval embedding practice; tagged for TEI usage. ⁴⁹	TEI (self-host) or HF Endpoint
Template semantic search embeddings	<code>intfloat/e5-large-v2</code>	MIT ⁵⁰	Model card states embedding size 1024; tagged for TEI. ⁵¹	TEI

Capability	Candidate model	License	Notes for Virtual Worker	Serving suggestion
Modern multilingual embeddings	Qwen/Qwen3-Embedding-0.6B	Apache-2.0 ⁵²	Model card states 0.6B params, 32k context, up to 1024 dims, supports instruction-aware embeddings. ⁵²	TEI
Zero-shot classification (intent classifier)	facebook/bart-large-mnli	MIT ⁵³	Trained on MNLI; often used for NLI-based zero-shot classification. ⁵⁴	HF Providers (fast start) or Endpoint
Summarization	facebook/bart-large-cnn	MIT ⁵⁵	Fine-tuned on CNN/DailyMail; documented summarization pipeline usage. ⁵⁵	HF Providers/Endpoint
NER extraction (entities like org/person/location)	dslim/bert-base-NER	MIT ⁵⁶	Fine-tuned on CoNLL-2003; recognizes LOC/ORG/PER/MISC. ⁵⁷	HF Providers/Endpoint

Template-to-n8n mapping design (naming conventions, parameter schemas, versioning)

A Virtual Worker becomes maintainable only if you treat workflows as **versioned products**.

Template entity - `template_key` (stable): e.g., `email.send`, `crm.upsert_contact`, `slack.post_summary` - `template_version` (SemVer): e.g., `1.4.0` - `n8n_workflow_id` and `n8n_workflow_version_id` (or immutable exported JSON hash) - `parameter_schema` (JSON Schema draft 2020-12) for strict validation. ⁵⁸

Recommended n8n workflow naming convention - `vw/<template_key>@<semver>`

Example: `vw/email.send@1.4.0`

Parameter schema strategy - Use JSON Schema 2020-12 to define required/optional parameters, enums, formats, and nested objects, so AI outputs can be validated before any side effects occur. ⁵⁹

- Use prompt templates that instruct the model to output **only** an object that validates against the schema; reject otherwise.

Approval flows (UX, webhook patterns, idempotency)

Approvals should be **native to the product** and **enforced in workflow execution**.

UX - "Preview plan" screen: what actions will be taken, which connectors, which targets. - "Approve / Reject / Edit parameters" with diff view between AI suggestion and user edits. - Audit capture: who approved, when, and what exactly.

n8n execution pattern - Use a Webhook-triggered workflow for task execution OR an API-triggered initiation from your backend. - Use the **Wait node** to pause execution until approval arrives (including "resume on webhook call"). n8n docs explain waiting/resuming on webhook as a core pattern. ⁶⁰
- Use **Respond to Webhook** or "When Last Node Finishes" modes to return results for synchronous requests; otherwise prefer async "respond immediately" and poll. ⁶¹

Idempotency - Use an idempotency key for: - Task creation (control plane) - Workflow dispatch (control plane → n8n) - Each side-effect step (e.g., "send email", "create ticket"), stored as `action_dedup_key`. - Stripe documents that idempotency works by caching the first result for a key and returning it for subsequent retries (even after failures), and recommends safe retries with backoff/jitter. ⁶

```
sequenceDiagram
    participant User
    participant API as Control Plane API
    participant AI as AI Orchestrator
    participant N8N as n8n Workflow
    participant Ext as External System

    User->>API: POST /tasks (Idempotency-Key)
    API->>AI: classify + pick template + extract params
    AI-->>API: template_key@version + params + risk score
    API-->>User: Show Plan + "Approve"

    User->>API: Approve (approval_id)
    API->>N8N: Start execution (task_run_id, action_dedup_key)
    N8N->>N8N: Wait node until approval received or pre-approved
    N8N->>Ext: Execute side-effect actions
    Ext-->>N8N: Result
    N8N-->>API: Completion webhook (task_run_id, outputs)
    API-->>User: Status + artifacts
```

Audit logging schema (explicit design)

Security logging and monitoring failures are recognized by OWASP as a major risk category; build audit logging as a first-class database model, not application "best effort." ⁶²

Suggested **append-only** audit table:

```
CREATE TABLE audit_log (
  id          UUID PRIMARY KEY,
  ts          TIMESTAMPTZ NOT NULL DEFAULT now(),
```

```

tenant_id      UUID NOT NULL,
actor_type     TEXT NOT NULL CHECK (actor_type IN ('user','system','api_key')),
actor_id       UUID,
action         TEXT NOT NULL, -- e.g., task.created, approval.granted,
workflow.dispatched
entity_type    TEXT NOT NULL, -- task, task_run, connector, template
entity_id      UUID NOT NULL,
request_id     TEXT,           -- correlate across services
idempotency_key TEXT,
ip             INET,
user_agent     TEXT,
metadata       JSONB NOT NULL -- structured details (diffs, params redacted)
);
CREATE INDEX ON audit_log (tenant_id, ts DESC);
CREATE INDEX ON audit_log (action, ts DESC);

```

Retry and error handling policies

Workflow error handling in n8n - n8n supports an Error Workflow pattern with the **Error Trigger** node: when a linked workflow fails, the error workflow receives details. ⁶³

- Combine this with control-plane retries and idempotency: if a workflow step times out or fails, retry safely only when the step is idempotent and guarded by a dedup key. ¹⁴

Timeouts - n8n documents execution timeout behavior; use global timeouts and per-step timeouts to avoid runaway executions. ⁶⁴

Concurrency - Use n8n concurrency control to avoid API rate-limit cascades and system unresponsiveness. ²⁶

Observability (logs, metrics, tracing)

Minimum viable observability stack: - **Metrics:** Prometheus + Grafana; n8n exposes `/metrics` when enabled via `N8N_METRICS=true`. ²⁵

- **Tracing:** instrument control-plane + AI service with OpenTelemetry (vendor-neutral framework for traces/metrics/logs). ⁶⁵

- **Logs:** Loki for cost-efficient log storage with chunks + indexes and tenant-separated log streams via tenant ID + labels. ⁶⁶

Testing strategy (unit, integration, e2e, load)

Testing must span three planes: control, workflow, inference.

- **Unit tests (control plane):** schema validation, idempotency table logic (replay semantics), RBAC checks, redaction rules.
- **Integration tests:** spin up ephemeral Postgres/Redis/n8n + stub external APIs; validate that a template execution produces expected side effects.

- **E2E tests:** drive UI → approvals → workflow completion; verify audit log entries exist and are correlated.
- **Load tests:** focus on queue mode (workers) and webhook scale; ensure concurrency caps and backpressure work. Queue mode architecture and concurrency controls are documented and should be validated under realistic load. ⁶⁷
- **Model eval tests:** run a golden set of tasks through extraction → JSON Schema validation pass rate; use `evaluate` or `lighteval` harness to track regressions. ⁶⁸

Project plan and delivery (phases 0–3)

Role definitions (owners)

To make ownership explicit in the plan: - **PM/PL** (Product lead): scope, roadmap, acceptance criteria, customer discovery - **TL** (Tech lead/Architect): architecture, non-functional requirements, reviews - **BE** (Backend engineer): API, DB, auth, orchestration logic - **FE** (Frontend engineer): UI, approvals, run history - **WF** (Workflow engineer): n8n templates, connector patterns, error workflows - **MLE** (ML engineer): model routing, eval harness, prompts, safety - **SRE** (Platform/SRE): deploy, observe, secure, on-call readiness - **Sec** (Security): threat model, RBAC, vault policies, reviews

Phase breakdown

Phase 0: foundations (Weeks 1–2)

Primary outcomes: decisions (licensing, tenancy), baseline infra, repo + CI, initial schemas, and a single “happy path” workflow running in dev.

Because this is a multi-tenant platform, Phase 0 must include a **licensing/edition decision** for n8n (Community vs Enterprise vs Embed) since many governance capabilities (Projects/RBAC, External Secrets, External Storage) aren't present in Community edition and n8n licensing restricts certain hosted/service use cases. ⁸

Phase 1: MVP (Weeks 3–6)

Primary outcomes: task intake UI/API, template registry, AI interpreter (template selection + param extraction), 3–5 templates, approvals, audit log, and basic observability.

Phase 2: scale and productization (Weeks 7–12)

Primary outcomes: queue-mode scale, webhook processors where needed, secrets vault integration, strong tenant isolation, standardized retry policies, more connectors/templates, and formal testing pipeline.

Phase 3: advanced autonomy (Weeks 13–16)

Primary outcomes: multi-step planning, tool selection, semi-autonomous execution under policy, improved evaluation and safety gating, and enterprise readiness.

Week-by-week plan (starting Monday, February 16, 2026)

Each week includes: why, how (tools/config), owner(s), dependencies, acceptance criteria.

Week (dates)	Deliverables (what/why)	How to execute (tools, config, commands)	Owners	Dependencies	Acceptance criteria
Week 1 (Feb 16–Feb 22)	Decision sprint: tenancy model + licensing path + security baseline. Needed because licensing and feature availability (RBAC, External Secrets, External Storage) affect architecture. ⁶⁹	Run threat modeling workshop using OWASP tenant isolation risks; define tenant boundary options; decide whether to use n8n Projects/RBAC (requires non-Community) or isolate via separate instances. ⁷⁰	PM/PL, TL, Sec	None	Written architecture decision record (ADR) + licensing decision draft; threat model doc with top risks + mitigations
Week 2 (Feb 23–Mar 1)	Dev environment live: control-plane skeleton + n8n queue-mode dev stack. Needed so all teams can iterate quickly and validate workflows early. ⁷¹	Implement Docker Compose stack; run <code>docker compose up -d</code> ; configure Postgres instead of SQLite (n8n default is SQLite otherwise). ⁷²	SRE, BE, WF	Week 1 decisions	Dev stack reachable; one workflow can run end-to-end in queue mode with a worker

Week (dates)	Deliverables (what/why)	How to execute (tools, config, commands)	Owners	Dependencies	Acceptance criteria
Week 3 (Mar 2–Mar 8)	Core data model + audit: tenants/users/tasks/task_runs/approvals/audit_log. Needed for idempotency, traceability, and multi-tenant correctness. ⁷³	Implement Postgres migrations; add idempotency key storage patterned after “cache first result.” ⁷⁴	BE, TL	Week 2	APIs create tasks idempotently; audit rows written for task.created and approval.created
Week 4 (Mar 9–Mar 15)	Task intake UI + API v1. Needed to collect real user tasks and start feedback loop.	Next.js UI: task form + list + details. Backend endpoints: <code>POST /tasks</code> , <code>GET /tasks/:id</code> .	FE, BE	Week 3	Users can create a task, see status “planned,” and view extracted parameters
Week 5 (Mar 16–Mar 22)	AI interpreter v1: template selection + parameter extraction + JSON Schema validation. Needed to map free text to repeatable templates. ⁷⁵	Build AI service that: embeddings search → rerank → LLM extracts params; validate with JSON Schema 2020-12. ⁵⁹	MLE, BE	Week 3–4	For 2 templates, ≥80% schema-valid extractions on a small golden set (target)

Week (dates)	Deliverables (what/why)	How to execute (tools, config, commands)	Owners	Dependencies	Acceptance criteria
Week 6 (Mar 23–Mar 29)	Approvals + execution handshake. Needed to prevent accidental side effects and support human-in-the-loop. Use n8n Wait on webhook call. ⁷⁶	Implement approval UI; n8n workflow pauses using Wait node and resumes on approval webhook; respond via Respond to Webhook or async update. ⁷⁷	FE, WF, BE	Week 4–5	A task requiring approval pauses, resumes, and completes; audit log captures approval.granted
Week 7 (Mar 30–Apr 5)	Template library v1 (3–5 templates) + connector priorities. Needed to demonstrate real ROI.	Build n8n workflows using HTTP Request node for APIs and dedicated nodes when available. ⁷⁸	WF, PM/PL	Week 6	3–5 templates shipped with docs; all templates versioned and schema-bound
Week 8 (Apr 6–Apr 12)	Error handling + retry-safe patterns. Needed because failures are inevitable; combine n8n Error Trigger + idempotency/backoff. ⁷⁹	Implement error workflows using Error Trigger; define retry policy with exponential backoff + jitter; enforce idempotency keys. ⁷⁹	WF, BE, SRE	Week 6–7	100% of workflow failures produce an error event; retries do not duplicate side effects
Week 9 (Apr 13–Apr 19)	Observability v1: metrics/logs/traces. Needed for production readiness and OWASP logging expectations. ⁸⁰	Enable <code>/metrics</code> with <code>N8N_METRICS=true</code> ; implement OpenTelemetry in backend; ship logs to Loki. ⁸¹	SRE, BE	Week 2	Dashboards show task throughput, failure rate, worker queue depth; trace a request across services

Week (dates)	Deliverables (what/why)	How to execute (tools, config, commands)	Owners	Dependencies	Acceptance criteria
Week 10 (Apr 20–Apr 26)	Security hardening sprint: RBAC/permissions, secret vault plan, n8n API disable/enable decisions. Needed for tenant isolation. ⁸²	If using n8n public API, configure X-N8N-API-KEY and scopes (enterprise). Otherwise disable API. ⁸³	Sec, TL, SRE	Weeks 1–9	Documented RBAC model; secrets never stored in plaintext; penetration-test checklist created
Week 11 (Apr 27–May 3)	Scale execution plane: add workers, optional webhook processors. Needed for load and latency. ⁶⁷	Implement autoscaling rules; separate main from webhook pool; tune concurrency via env/flags. ⁶⁷	SRE, WF	Week 9	Demonstrate stable throughput under load test; UI remains responsive
Week 12 (May 4–May 10)	Secrets vault + rotation. Needed to reduce blast radius and enable safe ops. Vault DB secrets engine supports dynamic credentials with TTL. ⁸⁴	Deploy Vault; configure DB secrets engine; rotate creds; integrate with app. ⁸⁵	SRE, Sec	Week 10	App uses dynamic DB creds; rotation procedure tested; vault audit logs enabled ⁸⁶

Week (dates)	Deliverables (what/why)	How to execute (tools, config, commands)	Owners	Dependencies	Acceptance criteria
Advanced autonomy v1:					
Week 13 (May 11–May 17)	multi-step plans with policy gates. Needed to increase value beyond single-step templates.	Add planner that proposes a sequence of templates; enforce approvals at each side-effect boundary; store plan graph.	MLE, TL, WF	Week 6+	Multi-step task executes with at least one approval gate and full audit trail
Evaluation + regression harness.					
Week 14 (May 18–May 24)	Needed to prevent model drift and prompt regressions. Use evaluate/lighteval. ⁶⁸	Build golden task suites; automated nightly eval; fail CI on regression thresholds.	MLE, BE	Week 5	Eval reports generated; regression gates enforced
Enterprise readiness:					
Week 15 (May 25–May 31)	private inference endpoints, network controls, multi-tenant infra posture review. HF supports AWS PrivateLink for endpoints. ³⁷	If using HF Endpoints, enable PrivateLink; else isolate self-hosted inference in private subnets/namespaces. ⁸⁷	SRE, Sec	Week 12	Private networking working; no public access to inference endpoints from internet
Launch readiness:					
Week 16 (Jun 1–Jun 7)	SLOs, incident runbooks, on-call, cost dashboards, template catalog v2.	Define SLOs; implement alerts; finalize connector roadmap; postmortem process.	PM/PL, SRE, TL	All	“Go-live checklist” complete; staging burn-in passed; rollback plan validated

Sprint plan for the first 10 weeks (5 sprints)

Each sprint is two weeks, aligned to the week plan above.

Sprint	Weeks	Deliverables	Owners	Acceptance tests
Sprint A	Weeks 1–2	ADRs + dev stack with n8n queue mode	TL, SRE	<code>docker compose up -d</code> yields healthy main+worker; first workflow executes in queue mode ⁸⁸
Sprint B	Weeks 3–4	Core schema + task intake UI/API	BE, FE	Idempotent task creation; audit rows created; UI shows tasks
Sprint C	Weeks 5–6	AI interpreter v1 + approvals + Wait-node execution	MLE, WF, FE	≥2 templates route correctly; approval pauses/resumes workflow via webhook wait ⁸⁹
Sprint D	Weeks 7–8	Template library v1 + error workflows + retry safety	WF, BE	Error Trigger workflow catches failures; retries don't duplicate actions ⁷⁹
Sprint E	Weeks 9–10	Observability v1 + security hardening	SRE, Sec	<code>/metrics</code> enabled; traces available; API disabled if unused ⁹⁰

Backlog: epics → user stories → acceptance tests

Below is a practical initial backlog that aligns with the plan.

Epic	User story	Acceptance test
Tenant & identity	As an org admin, I can create a tenant and invite users	Creating a tenant creates isolated data; invited user cannot read other tenant tasks (negative test)
Task intake	As a user, I can submit a task with an idempotency key	Re-sending same request with same key returns same task id/result ⁷⁴
Template registry	As a platform owner, I can publish versioned templates with parameter schemas	Template <code>@version</code> cannot be overwritten; parameters validate via JSON Schema 2020-12 ⁵⁹
AI interpretation	As a user, I can submit free text and get a structured plan mapped to a template	For golden tests, ≥X% top-1 template accuracy; ≥Y% schema-valid extraction

Epic	User story	Acceptance test
Approvals	As a manager, I can require approvals for risky actions	Workflow pauses (Wait node) and resumes only after approval webhook; audit record exists ⁹¹
Workflow execution	As a user, I can run a task and see status transitions	Status: planned→approved→running→succeeded/failed; outputs stored
Error handling	As an operator, I get alerted and errors are correlated	Error workflow triggers on failure; alert fired; trace links to request id ⁹²
Observability	As SRE, I can see platform health dashboards	Prometheus scrapes <code>/metrics</code> ; Loki query finds task_run logs by request id ⁹³
Secrets & connectors	As a tenant admin, I can connect my SaaS credentials safely	Secrets stored only in vault/secret manager; rotated successfully ⁹⁴
Inference governance	As a platform owner, I can choose hosted vs self-hosted inference per tenant	Tenant policy routes calls to Providers/Endpoints/self-host; private endpoint supported ⁹⁵

Connector priorities (recommended starting set)

Select connectors by (a) customer ROI, (b) reliability, (c) security surface area, and (d) time-to-implement.

A practical phased list: - Tier 0: HTTP Request + Webhook (universal coverage) because n8n's HTTP Request node can call any REST API, and Webhook can accept inbound triggers. ⁹⁶

- Tier 1: Email (Gmail/SMTP), Slack/Teams, Google Sheets, Jira/Linear, HubSpot/Salesforce (depends on customer base) - Tier 2: Data warehouses (Snowflake/BigQuery), ticketing (Zendesk), file storage (Drive/S3), CRM enrichment

Risk register and mitigations

Risk	Why it matters	Mitigation
Licensing mismatch with business model	n8n license/edition restrictions can block productization and required features	Resolve in Week 1; engage n8n for Embed/Enterprise if hosting customer workflows/credentials ⁹⁷
Cross-tenant leakage	Single bug can expose multiple customers	Tenant-aware middleware everywhere; DB isolation strategy; rigorous tests; OWASP multi-tenant guidance ⁹⁸
Unsafe retries causing duplicate actions	Can spam users or corrupt business systems	Idempotency keys + safe retries with jitter/backoff; dedup table per side effect ⁶

Risk	Why it matters	Mitigation
n8n code execution vulnerabilities	Workflow editing permissions can be high risk; CVEs show “workflow modify” can lead to host command execution	Minimize who can edit workflows; isolate n8n; prefer hardened task runner setup; patch quickly ⁹⁹
Observability gaps	Without logs/metrics/traces, incident response is slow (OWASP A09)	Standardize OpenTelemetry + Prometheus + Loki early ¹⁰⁰
GPU cost blowups	Model choices can dominate spend	Start with serverless/providers; add rate limits + caching; move heavy tenants to dedicated endpoints or self-host ¹⁰¹

¹ ² ¹⁰ ¹² ¹⁵ ²⁰ ²² ⁶⁷ ⁷¹ **Configuring queue mode | n8n Docs**

<https://docs.n8n.io/hosting/scaling/queue-mode/>

³ ¹¹ ¹⁶ ¹⁹ ⁹⁵ ¹⁰¹ **Inference Providers - Hugging Face**

https://huggingface.co/docs/inference-providers/en/index?utm_source=chatgpt.com

⁴ ³⁵ **About Inference Endpoints - Hugging Face**

https://huggingface.co/docs/inference-endpoints/about?utm_source=chatgpt.com

⁵ ³⁸ ⁴⁷ **Quick Tour - Hugging Face**

https://huggingface.co/docs/text-generation-inference/quicktour?utm_source=chatgpt.com

⁶ ⁴² ⁷⁴ **Idempotent requests | Stripe API Reference**

https://docs.stripe.com/api/idempotent_requests?lang=python&ref=...&utm_source=chatgpt.com

⁷ ⁷⁰ ⁹⁸ **Multi Tenant Security - OWASP Cheat Sheet Series**

https://cheatsheetseries.owasp.org/cheatsheets/Multi_Tenant_Security_Cheat_Sheet.html?utm_source=chatgpt.com

⁸ ⁹ ⁶⁹ ⁹⁷ **Sustainable Use License | n8n Docs**

https://docs.n8n.io/sustainable-use-license/?utm_source=chatgpt.com

¹³ **Webhook node documentation | n8n Docs**

https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.webhook/?utm_source=chatgpt.com

¹⁴ **Designing robust and predictable APIs with idempotency - Stripe**

https://stripe.com/blog/idempotency?utm_source=chatgpt.com

¹⁷ ⁸⁴ ⁹⁴ **Database secrets engine - Vault | HashiCorp Developer**

https://developer.hashicorp.com/vault/docs/secrets/databases?utm_source=chatgpt.com

¹⁸ ⁶² ⁷³ ⁸⁰ ¹⁰⁰ **A09 Security Logging and Monitoring Failures - OWASP Top 10:2021**

https://owasp.org/Top10/2021/A09_2021-Security_Logging_and_Monitoring_Failures/?utm_source=chatgpt.com

²¹ **Compose file reference | Docker Docs**

https://docs.docker.com/reference/compose-file/?utm_source=chatgpt.com

²³ **Set a custom encryption key | n8n Docs**

https://docs.n8n.io/hosting/configuration/configuration-examples/encryption-key/?utm_source=chatgpt.com

24 **Disable the public REST API | n8n Docs**

https://docs.n8n.io/hosting/securing/disable-public-api/?utm_source=chatgpt.com

25 81 90 93 **Enable Prometheus metrics | n8n Docs**

https://docs.n8n.io/hosting/configuration/configuration-examples/prometheus/?utm_source=chatgpt.com

26 **Concurrency control | n8n Docs**

https://docs.n8n.io/hosting/scaling/concurrency-control/?utm_source=chatgpt.com

27 88 **docker compose up | Docker Docs - Docker Documentation**

https://docs.docker.com/reference/cli/docker/compose/up/?utm_source=chatgpt.com

28 43 **Binary data environment variables | n8n Docs**

<https://docs.n8n.io/hosting/configuration/environment-variables/binary-data/>

29 **External storage for binary data | n8n Docs**

<https://docs.n8n.io/hosting/scaling/external-storage/>

30 **External data storage environment variables | n8n Docs**

<https://docs.n8n.io/hosting/configuration/environment-variables/external-data-storage/>

31 **Role-based access control (RBAC) | n8n Docs**

https://docs.n8n.io/user-management/rbac/?utm_source=chatgpt.com

32 **External secrets | n8n Docs**

https://docs.n8n.io/external-secrets/?utm_source=chatgpt.com

33 83 **Authentication | n8n Docs**

https://docs.n8n.io/api/authentication/?utm_source=chatgpt.com

34 **n8n Embed Documentation and Guides | n8n Docs**

https://docs.n8n.io/embed/?utm_source=chatgpt.com

36 **Inference Endpoints**

https://huggingface.co/docs/huggingface_hub/en/guides/inference_endpoints

37 **Create a Private Endpoint with AWS PrivateLink - Hugging Face**

https://huggingface.co/docs/inference-endpoints/guides/private_link?utm_source=chatgpt.com

39 **huggingface/text-generation-inference - GitHub**

https://github.com/huggingface/text-generation-inference?utm_source=chatgpt.com

40 41 **Quick Tour - Hugging Face**

https://huggingface.co/docs/text-embeddings-inference/quick_tour?utm_source=chatgpt.com

44 68 **Evaluate on the Hub - Hugging Face**

https://huggingface.co/docs/evaluate/index?utm_source=chatgpt.com

45 **Lighteval - Hugging Face**

https://huggingface.co/docs/lighteval/index?utm_source=chatgpt.com

46 **HuggingFaceH4/zephyr-7b-beta · Hugging Face**

<https://huggingface.co/HuggingFaceH4/zephyr-7b-beta>

48 **mistralai/Mistral-7B-Instruct-v0.3 · Hugging Face**

<https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.3>

- 49 **README.md · BAAI/bge-base-en-v1.5 at main - Hugging Face**
https://huggingface.co/BAAI/bge-base-en-v1.5/blob/main/README.md?utm_source=chatgpt.com
- 50 **intfloat/e5-large-v2 at main - Hugging Face**
https://huggingface.co/intfloat/e5-large-v2/tree/main?utm_source=chatgpt.com
- 51 **intfloat/e5-large-v2 · Hugging Face**
https://huggingface.co/intfloat/e5-large-v2?utm_source=chatgpt.com
- 52 **Qwen/Qwen3-Embedding-0.6B · Hugging Face**
<https://huggingface.co/Qwen/Qwen3-Embedding-0.6B>
- 53 **facebook/bart-large-mnli · Hugging Face**
https://huggingface.co/facebook/bart-large-mnli?ref=datafantic&utm_source=chatgpt.com
- 54 **facebook/bart-large-mnli · Hugging Face**
https://huggingface.co/facebook/bart-large-mnli?utm_source=chatgpt.com
- 55 **facebook/bart-large-cnn · Hugging Face**
<https://huggingface.co/facebook/bart-large-cnn>
- 56 **dslim/bert-base-NER at main - Hugging Face**
https://huggingface.co/dslim/bert-base-NER/tree/main?utm_source=chatgpt.com
- 57 **dslim/bert-base-NER · Hugging Face**
https://huggingface.co/dslim/bert-base-NER?utm_source=chatgpt.com
- 58 **JSON Schema - Specification [#section]**
https://json-schema.org/specification?utm_source=chatgpt.com
- 59 75 **Draft 2020-12 - json-schema.org**
https://json-schema.org/draft/2020-12?utm_source=chatgpt.com
- 60 76 **Waiting | n8n Docs**
https://docs.n8n.io/flow-logic/waiting/?utm_source=chatgpt.com
- 61 77 **Respond to Webhook | n8n Docs**
https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.respondtwebhook/?utm_source=chatgpt.com
- 63 79 92 **Error Trigger node documentation | n8n Docs**
https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.errortrigger/?utm_source=chatgpt.com
- 64 **Configure workflow timeout settings | n8n Docs**
https://docs.n8n.io/hosting/configuration/configuration-examples/execution-timeout/?utm_source=chatgpt.com
- 65 **Documentation - OpenTelemetry**
https://opentelemetry.io/docs/?utm_source=chatgpt.com
- 66 **Manage storage | Grafana Loki documentation**
https://grafana.com/docs/loki/latest/operations/storage/?utm_source=chatgpt.com
- 72 **Supported databases and settings | n8n Docs**
https://docs.n8n.io/hosting/configuration/supported-databases-settings/?utm_source=chatgpt.com
- 78 96 **HTTP Request node documentation | n8n Docs**
https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.httprequest/?utm_source=chatgpt.com

82 RBAC projects | n8n Docs

https://docs.n8n.io/user-management/rbac/projects/?utm_source=chatgpt.com

85 Dynamic secrets for database credential management | Vault | HashiCorp ...

https://developer.hashicorp.com/vault/tutorials/db-credentials/database-secrets?utm_source=chatgpt.com

86 Vault product documentation | Vault | HashiCorp Developer

https://developer.hashicorp.com/vault/docs?utm_source=chatgpt.com

87 Multi-tenancy - Kubernetes

https://kubernetes.io/docs/concepts/security/multi-tenancy/?utm_source=chatgpt.com

89 91 n8n-docs/docs/integrations/builtin/core-nodes/n8n-nodes-base.wait.md at ...

[https://github.com/n8n-io/n8n-docs/blob/main/docs/integrations/builtin/core-nodes/n8n-nodes-base.wait.md?](https://github.com/n8n-io/n8n-docs/blob/main/docs/integrations/builtin/core-nodes/n8n-nodes-base.wait.md?utm_source=chatgpt.com)
[utm_source=chatgpt.com](https://github.com/n8n-io/n8n-docs/blob/main/docs/integrations/builtin/core-nodes/n8n-nodes-base.wait.md?utm_source=chatgpt.com)

99 Arbitrary Command Execution in Pyodide based Python Code Node ... - GitHub

https://github.com/n8n-io/n8n/security/advisories/GHSA-62r4-hw23-cc8v?utm_source=chatgpt.com